# Identifying Security Vulnerabilities in Kubernetes Environments

Author: Patrick Trecek
Advisor: *Lenny Zeltser*

Accepted: January 13. 2026

Abstract

This research aims to develop a practical methodology for identifying security misconfigurations in Kubernetes environments, across both Infrastructure-as-Code (IaC) and live cluster states. Using Amazon Elastic Kubernetes Service (EKS), ten intentionally vulnerable Kubernetes manifests that aligned with the OWASP Kubernetes Top 10 were deployed to support controlled experimentation. Five test scenarios assess tool performance across repository and live-cluster scans. These scenarios illustrate how combining multiple scanning approaches can provide broader visibility into environmental risk. The findings highlight the importance of a multi-tool strategy and of continuously validating both code and cluster state to maintain a secure Kubernetes environment.

# 1. Introduction

OS versions, dependency libraries, and updates can vary across hosts; containerization enables applications to be bundled with all their dependencies, streamlining deployment. Containers have become a staple in the IT industry, with 92% of respondents reporting container use (Wilson, Diachkova, & Floyd, 2025). With the widespread adoption of containerized applications, developers require a method to track and manage their containers. In June 2014, Google released an open-source project called Kubernetes (DZone, 2020). Kubernetes is an open-source container orchestration system that manages containerized applications across multiple hosts, supporting deployment, monitoring, and scaling (Hohn, 2018).

The trend shows more organizations adopting containers and Kubernetes. This fast-deployment approach can quickly get a company up and running, hosting a variety of applications to support its business. It is reported that 60% of enterprises have adopted Kubernetes (Cloud Native Computing Foundation, 2024). As it becomes faster and more scalable, the risk of overlooking security could be real. According to a Red Hat State of Kubernetes security report, 60% of respondents worry about vulnerabilities, misconfigurations, and exposures in their container and Kubernetes environments (Red Hat, 2023).

The risks of an insecure cluster cannot be overlooked. These risks may include compromised cloud accounts, leaked intellectual property, cluster-wide denial-of-service attacks, and cryptocurrency mining. These risks are not limited to smaller organizations. For example, in 2018, an unsecured Kubernetes administration console and insecure AWS credentials in configuration files led to a compromise of the well-known company Tesla. Attackers pivoted to the cloud environment and installed cryptocurrency-mining software on cloud servers (Eleweke, 2025). Many similar security breaches have stemmed from simple misconfigurations like this. With this new technology in play, there may be an increased need for a systematic approach to securing the environment.

The primary purpose of container orchestration is to automate the desired state of a cluster. The desired state is defined via Infrastructure-as-Code in a version-controlled

repository, while the actual state is the running Kubernetes cluster. This study evaluates the effectiveness of security scanning tools in identifying Kubernetes misconfiguration across both states. The research proposes and applies a repeatable methodology to assess how repository and live cluster scans detect injected vulnerabilities that map to OWASP. By comparing detection precision and consistency, this work aims to demonstrate the multi-layered security validation approach for maintaining secure Kubernetes environments.

## 2. Research Method

In reference to the OWASP Top 10 Kubernetes security risks, the study has recreated YAML (Yet Another Markup Language) files for each vulnerability. According to OWASP, as of this paper, the 2025 list is still in progress. The most recent update is from 2022, as there were no public releases in 2023 or 2024. These are the most up-to-date OWASP Top 10 Kubernetes Risks (OWASP Foundation, 2025).

- K01: Insecure Workload Configurations
- K02: Supply Chain Vulnerabilities
- K03: Overly Permissive RBAC Configurations
- K04: Lack of Centralized Policy Enforcement
- K05: Inadequate Logging and Monitoring
- K06: Broken Authentication Mechanisms
- K07: Missing Network Segmentation Controls
- K08: Secrets Management Failures
- K09: Misconfigured Cluster Components
- K10: Outdated and Vulnerable Kubernetes Components

Kubernetes utilizes a declarative model in which the user defines the desired state of a resource, and the system maintains that configuration. Generally, YAML files describe the Kubernetes resource the team wants to deploy to the cluster, and Kubernetes compares its current state to the desired state defined in the YAML. This study injected security vulnerabilities into the cluster with the kubectl command. Kubectl is a

command-line tool for configuring and interacting with a Kubernetes cluster. It talks to the Kubernetes API server based on the context stored in the kubeconfig file. (Mocevicius, 2020).

## 2.1 Vulnerability Injections

A folder in the repository contained YAML files that define poorly configured Kubernetes resources, as shown by Figure 1.

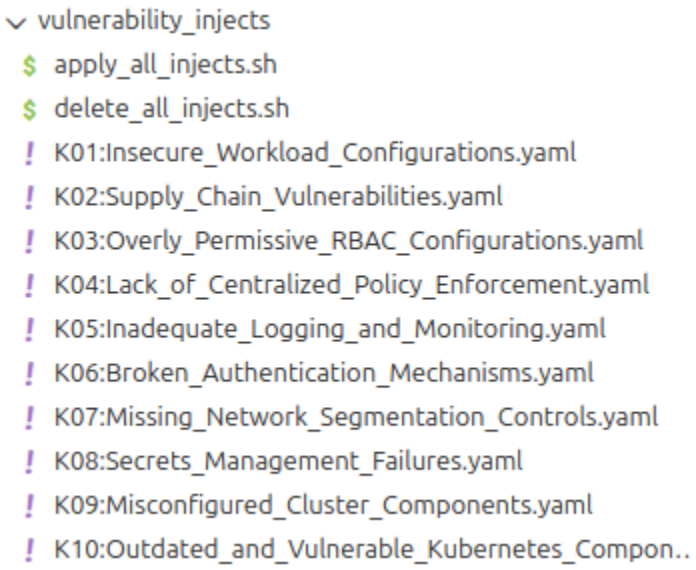



Figure 1: Vulnerability Inject YAML files

Figures 2, 3, and 4 are examples of misconfigured YAML files. The beginning text will highlight where the vulnerability falls within the OWASP top 10 Kubernetes risks, e.g., K01. As illustrated in the figures, K03 introduces a service account with unrestricted, cluster-wide administrative privileges, K08 presents a ConfigMap containing hardcoded credentials, and K02 deploys a vulnerable container image sourced from Vulhub. The study provided one branch of the repository that contained all vulnerability injections and another that did not. This simulated whether the injections are in place for the repository scan. For live cluster scans, vulnerabilities were applied to the cluster using the kubectl apply command post-deployment, as shown below.

**kubectl apply -f <vulnerability_inject.yaml>**

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: rbac-test-sa
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: rbac-test-binding
roleRef:
  kind: ClusterRole
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
subjects:
  - kind: ServiceAccount
    name: rbac-test-sa
    namespace: default
```

Figure 2: K03 Overly Permissive RBAC Example

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: insecure-config
data:
  db_password: "SuperSecret123"
  api_key: "ABCDEF123456"
```

Figure 3: K08 Secrets Management Failures Example

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vuln-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vuln
  template:
    metadata:
      labels:
        app: vuln
    spec:
      containers:
        - name: vuln
          image: vulhub/pgadmin:6.16    # <-- known vulnerable version
```

Figure 4: K02 Supply Chain Vulnerabilities

## 2.2 Scanning Tools

Three standard Kubernetes security tools were chosen: Kubescape, Kube-Bench, and Checkov. Wiz has identified these open-source tools among its top 11 Kubernetes security tools (Wiz, Inc. 2025).

- Kubescape is a popular tool that seems to do it all; it can scan the live cluster and integrates with popular IDEs to scan code or repositories.
- Kube-Bench validates your live cluster against CIS controls. CIS (Center for Internet Security) controls are a prescriptive, prioritized, and simplified set of best practices to strengthen your cybersecurity posture (Center for Internet Security, n.d.).
- Checkov scans infrastructure as code, but is not used to scan live clusters.

It is essential to recognize the different goals of these tools and whether they are intended to scan IaC, a live cluster, or both. Ideally, the desired and actual states should match, but this is not always the case. Configuration drift is a problem that many organizations face when running workloads across multiple clusters. Forty

percent of Kubernetes users report that configuration drift negatively impacts the stability of their environment (Shwartz, 2025). Configuration drift occurs when the actual running state of a system (e.g., a Kubernetes cluster) gradually diverges from the intended or declared state. This is generally considered poor practice, but it can occur if developers make quick patches or add additional Kubernetes resources after deployment.

# 3. Test Environment

The environment used a standard AWS service, Elastic Kubernetes Service (EKS), to host the Kubernetes cluster. Terraform was used to build the AWS cloud environment. Terraform is an infrastructure-as-code tool that lets you build, change, and version cloud resources safely and efficiently (HashiCorp, Inc., n.d.). After applying Terraform, a setup script was executed to host applications on the cluster in the lab environment. The setup script utilized Helm charts to deploy applications to the cluster (Nginx, Minio, Grafana/Prometheus, and pgAdmin/PostgreSQL). The goal was to make the scans more realistic by having other Kubernetes resources in play before injecting the vulnerabilities. The control used was the repository without the injects, using Terraform and script to set up the EKS cluster and host the applications.

## 3.1 Test Scenarios

The following provides each scenario and its relevant guidance.

Scenario 1: Use Kubescape & Checkov to scan the default repository without the vulnerability injects present, and the injects were not applied to the live cluster. Then Kubescape & Kube-Bench were used to scan the live cluster. This provided a baseline to work from and a real-world scenario, as Helm charts were deployed as-is with their values file.

Scenario 2: Use Kubescape & Checkov to scan the default repository that contained the vulnerability injections. However, the vulnerabilities were not applied to the live cluster. Kubescape & Kube-bench were then used to scan the live cluster. This scenario highlights insecure code in the repository that is not applied to the live cluster.

Scenario 3: Use Kubescape & Checkov to scan the default repository that contained the vulnerability injections , and those injections were applied to the live cluster. Then, Kubescape & Kube-Bench were used to scan the live cluster.

Scenario 4: Use Kubescape & Checkov to scan the default repository, which did not contain vulnerability injections. Mimicking configuration drift, the vulnerability injections were applied to the live cluster. Then Kubescape & Kube-Bench were used to scan the live cluster.

Scenario 5: Incorporated Kubescape & Checkov jobs into a Git workflow to run scans when code was pushed to the repository. Each code push was one of the YAML vulnerability injections. Results were observed on the ability to push the security vulnerabilities to the repository. Kubescape & Kube-bench were then used to scan the live cluster.

## 3.2 Scans

To evaluate the cluster's desired state security posture, repository-based scans were performed on the code prior to deployment. These scans assess code intent and identify security misconfigurations before being applied to the live Kubernetes cluster. The following commands initiate scans to analyze Kubernetes manifests, Helm charts, and related configuration files.

**kubescape scan SANS-White-Paper-EKS -v**

The first scan above will provide an overview of the repository's security posture, highlighting concerns, and will conclude with a MITRE/NSA compliance score.

**kubescape scan framework nsa SANS-White-Paper-EKS -v**
**kubescape scan framework mitre SANS-White-Paper-EKS -v**

The two scans above will provide results from the NSA and MITRE frameworks, respectively.

**checkov -d SANS-White-Paper-EKS**

Checkov scans infrastructure-as-code and configuration files, such as Terraform or Kubernetes YAML, and returns results from the security checks. The output starts with a unique identifier, followed by a pass or fail result.
e.g. CKV_K8S_14 as part of its Kubernetes checks

**Kubescape VSCode extension**

With the Kubescape VSCode extension, when a repository is saved locally in VSCode, the extension will scan the saved file. Results will appear as problems in the VSCode terminal if Kubescape finds security concerns.

To evaluate the cluster's actual state, live cluster scans were performed against the running Kubernetes environment. These scans capture runtime context and configuration drift not visible in repository-based analysis. The following commands were run against the live cluster.

**kubescape scan -v**

The above scan checks the configuration of a live cluster against a library of security policies called controls. The results will be similar to the Kubescape scan against a repository, with additional add-ins such as the results of the actual Kubernetes control plane

**kube-bench**

```
kubectl create namespace kube-bench

kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job-eks.yaml -n kube-bench
```

The Kube-Bench scan ran as a job within our Kubernetes cluster. A job was used that was designed for EKS, as defined in the job-eks.yaml file. To retrieve the scan results, the logs were obtained from the pod created by the YAML file. These controls are organized into categories that cover different parts of the Kubernetes system (Pillai, 2024).

1. Control Plane Components: Focuses on securing the API server, Controller Manager, and Scheduler.
2. etcd: Controls for securing the cluster's distributed key-value store.
3. Worker Node Components: Controls for securing the Kubelet and other node components.
4. Policies: Controls for Pod Security Policies (PSPs, though deprecated), Network Policies, and general RBAC.
5. Managed Services: Controls specific to managed Kubernetes offerings like EKS, GKE, and AKS.

Note: kube-bench will skip Section 1, Control Plane Components, because this will be managed by AWS

# 4. Findings and Discussion

This section presents and analyzes the results of security scans from both repository-based and live cluster environments. The findings are organized into a series of controlled scenarios that were designed to evaluate different security tools under varying conditions. The results are discussed in the context of detection coverage, consistency between desired and actual states, and proper scan identification of the OWASP Kubernetes top 10.

## 4.1 Scenario 1- No vulnerabilities injected in the repository or applied to the cluster

For the first scenario, the study used a separate branch of the repository that did not contain any vulnerability-injected YAML files. After repository-based scans were concluded, no vulnerability injects were applied to the live cluster.

**kubescape scan SANS-White-Paper-EKS**

```
Security posture overview for repo: 'SANS-White-Paper-EKS'

Workload
```

| Control name | Resources | View details |
|---|---|---|
| Host PID/IPC privileges | 1 | $ kubescape scan control C-0038 SANS-White-Paper-EKS -v |
| HostNetwork access | 1 | $ kubescape scan control C-0041 SANS-White-Paper-EKS -v |
| HostPath mount | 1 | $ kubescape scan control C-0048 SANS-White-Paper-EKS -v |
| Non-root containers | 6 | $ kubescape scan control C-0013 SANS-White-Paper-EKS -v |

Figure 5: Kubescape scan summary for code repo

Without the vulnerability injections, the Kubescape scan shows that deploying the out-of-the-box Helm charts results in a few at-risk misconfigurations. Figure 5 reveals the summary results from the repository scan. These indicate control security failures, such as host-level privileges and root containers.

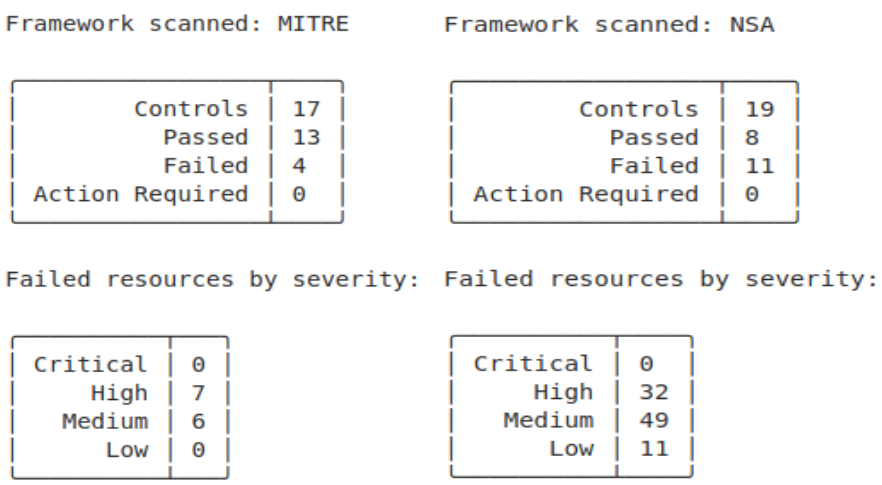



Figure 6: Kubescape scan against code repo using NSA & Mitre controls

MITRE and NSA scans showed 4 and 11 failed controls, respectively, as seen in Figure 6.

Checkov scan of repository

**checkov -d projects/SANS-White-Paper-EKS**

Checkov will display scan results in different sections, such as Terraform, Kubernetes, and secrets. The Terraform code showed three total failures against CKV_TF_1: Ensuring Terraform module sources use a commit hash (all from the same file, three different resources). As previously discussed, Terraform was used to construct the AWS environment for the tests.

```
terraform scan results:

Passed checks: 4, Failed checks: 3, Skipped checks: 0
```

Figure 7: Checkov summary Terraform results against code repo

The Checkov Kubernetes scan revealed 24 total failures.

```
kubernetes scan results:

Passed checks: 69, Failed checks: 24, Skipped checks: 0
```

Figure 8: Checkov summary Kubernetes results against code repo

These failures included:

CKV_K8S_21: "The default namespace should not be used"

CKV_K8S_28: "Minimize the admission of containers with the NET_RAW capability"

CKV_K8S_9: "Readiness Probe Should be Configured"

KV_K8S_30: "Apply security context to your containers"

CKV_K8S_38: "Ensure that Service Account Tokens are only mounted where necessary"

CKV_K8S_31: "Ensure that the seccomp profile is set to docker/default or runtime/default"

CKV_K8S_10: "CPU requests should be set"

CKV_K8S_29: "Apply security context to your pods and containers"

CKV_K8S_22: "Use read-only filesystem for containers where possible"

CKV_K8S_37: "Minimize the admission of containers with capabilities assigned"

CKV_K8S_13: "Memory limits should be set"

CKV_K8S_12: "Memory requests should be set"

CKV_K8S_40: "Containers should run as a high UID to avoid host conflict"

CKV_K8S_8: "Liveness Probe Should be Configured"

CKV_K8S_20: "Containers should not run with allowPrivilegeEscalation"

KV_K8S_14: "Image Tag should be fixed - not latest or blank"

CKV_K8S_11: "CPU limits should be set"

CKV_K8S_23: "Minimize the admission of root containers"

CKV_K8S_43: "Image should use digest"

The Checkov secrets scan showed failures for CKV_SECRET_6: "Base64 High Entropy String"

```
secrets scan results:

Passed checks: 0, Failed checks: 9, Skipped checks: 0
```

Figure 9: Checkov summary secrets scan results against code repo

Initial observations showed that, out of the box, deploying the Helm charts and quickly getting the applications up and running in the lab led to many security failures. This is not uncommon in industry. Recent research shows that Helm chart misconfigurations are widespread. In an automated analysis of the 60 most widely used public Helm charts, Minna et al. (2025) found recurring security policy violations that weaken workload isolation and increase the likelihood of exploitable Kubernetes misconfigurations. The presence of these misconfigurations, per this study, implies a risk that secrets may also be mishandled. Basak et al. (2023) collected a dataset of 97,479 secrets from 818 public GitHub repos. They found that secret exposure increased two-fold from 2021 to 2020.

Kubescape against the live cluster:

Access control

| Control name | Resources | View details |
|---|---|---|
| Administrative Roles | 0 | $ kubescape scan control C-0035 -v |
| List Kubernetes secrets | 2 | $ kubescape scan control C-0015 -v |
| Minimize access to create pods | 0 | $ kubescape scan control C-0188 -v |
| Minimize wildcard use in Roles and ClusterRoles | 0 | $ kubescape scan control C-0187 -v |
| Portforwarding privileges | 0 | $ kubescape scan control C-0063 -v |
| Prevent containers from allowing command execution | 0 | $ kubescape scan control C-0002 -v |
| Roles with delete capabilities | 1 | $ kubescape scan control C-0007 -v |
| Validate admission controller (mutating) | 2 | $ kubescape scan control C-0039 -v |
| Validate admission controller (validating) | 1 | $ kubescape scan control C-0036 -v |

Secrets

| Control name | Resources | View details |
|---|---|---|
| Applications credentials in configuration files | 1 | $ kubescape scan control C-0012 -v |

Network

| Control name | Resources | View details |
|---|---|---|
| Missing network policy | 7 | $ kubescape scan control C-0260 -v |

Workload

| Control name | Resources | View details |
|---|---|---|
| Host PID/IPC privileges | 1 | $ kubescape scan control C-0038 -v |
| HostNetwork access | 1 | $ kubescape scan control C-0041 -v |
| HostPath mount | 1 | $ kubescape scan control C-0048 -v |
| Non-root containers | 1 | $ kubescape scan control C-0013 -v |
| Privileged container | 0 | $ kubescape scan control C-0057 -v |

Figure 10: Kubescape scan against live cluster

Figure 10 reveals the results from the Kubescape live cluster scan, this included results from the control plane. For example, a failure for secret/etcd encryption enabled means anyone with etcd access can read plaintext secrets. To view the specific resource that is flagged, users will need to drill down and run a targeted scan on the control. For example: **kubescape scan control C-0035 -v**

The overall MITRE & NSA compliance scores are below

- MITRE: 75.71%
- NSA: 71.42%

The Kube-Bench scans of a live cluster are detailed in Figures 11 and 12:

Figure 11 shows the Kube-Bench pod created in the cluster, which contained the necessary logs of the results.

```
NAME                READY   STATUS      RESTARTS   AGE
kube-bench-9727g    0/1     Completed   0          126m
```

Figure 11: Kube-Bench pod created in the cluster

```
Kubectl logs kube-bench-w6x5v -n kube-bench
```

```
== Summary total ==
12 checks PASS
1 checks FAIL
33 checks WARN
0 checks INFO
```

Figure 12: Kube-Bench summary results

The Kube-Bench job checked 46 CIS controls. There was one fail and numerous warnings from Kube-Bench. The failure seen in Figure 13 was related to K8s event recording; however, our nodes are managed by EKS node groups, and AWS controls the kubelet.service file where the flag is set. The failure is related to using Amazon EKS instead of a self-managed cluster.

```
== Remediations node ==
3.2.7 If using a Kubelet config file, edit the file to set eventRecordQPS: to an appropriate
level.
If using command line arguments, edit the kubelet service file
/etc/systemd/system/kubelet.service.d/10-kubeadm.conf on each worker node
and set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable.
Based on your system, restart the kubelet service. For example:
systemctl daemon-reload
systemctl restart kubelet.service
```

Figure 13: CIS control 3.2.7 was the one fail

Scenario 1 revealed that even without intentionally injected vulnerabilities, default Helm chart deployments produced numerous security misconfigurations. The security tools identified issues such as root containers, missing security context settings, unpinned images, weak namespace usage, and plaintext secrets. These findings demonstrate that "out-of-the-box" Helm chart deployments may be inherently insecure and require hardening and further validation.

## 4.2 Scenario 2 – Vulnerabilities injected into the repository, but not applied to cluster

For scenario two, the vulnerability injections were located inside the repository. A scan took place prior to deployment. The vulnerabilities were not applied to the live cluster, and the live scan took place.

Figure 14 shows the Kubescape scan run against the repository containing the vulnerability-injects folder.

```
Security posture overview for repo: 'SANS-White-Paper-EKS'

Workload
```

| Control name | Resources | View details |
|---|---|---|
| Host PID/IPC privileges | 2 | $ kubescape scan control C-0038 SANS-White-Paper-EKS -v |
| HostNetwork access | 1 | $ kubescape scan control C-0041 SANS-White-Paper-EKS -v |
| HostPath mount | 2 | $ kubescape scan control C-0048 SANS-White-Paper-EKS -v |
| Non-root containers | 12 | $ kubescape scan control C-0013 SANS-White-Paper-EKS -v |
| Privileged container | 2 | $ kubescape scan control C-0057 SANS-White-Paper-EKS -v |

Figure 14: Kubescape scan summary for code repo

Running Kubescape against the repository shows security controls that failed in the repository's Kubernetes YAML manifests. A total of 19 resources were flagged as

failures across control categories, including Host PID/IPC privileges, HostPath mount, and Privileged container controls. Drilling down into these failed resources uncovered 6 of the 10 injects flagged. These include K01, K02, K04, K05, K07, and K09. Most of the failures came from the non-root container control. This does not pinpoint the exact OWASP security risk attempted to be replicated. Nonetheless, 60% of the injects were identified by the scan. Figure 15 shows an example of a flagged resource.

```
Source: vulnerability_injects/K09:Misconfigured_Cluster_Components.yaml
ApiVersion: v1
Kind: Pod
Name: kubelet-placeholder

Controls: 1 (Failed: 1, action required: 0)

┌──────────┬──────────────────┬──────────────────────────────┐
│ Severity │ Control name     │ Docs                         │
├──────────┼──────────────────┼──────────────────────────────┤
│ Medium   │ Non-root containers │ https://hub.armosec.io/docs/c-0013 │
└──────────┴──────────────────┴──────────────────────────────┘
```

Figure 15: Example Kubescape drill down revealing security inject

```
Framework scanned: MITRE
┌────────────────┬────┐
│       Controls │ 17 │
│         Passed │ 11 │
│         Failed │ 6  │
│ Action Required │ 0  │
└────────────────┴────┘

Failed resources by severity:
┌──────────┬────┐
│ Critical │ 0  │
│     High │ 10 │
│   Medium │ 6  │
│      Low │ 0  │
└──────────┴────┘

Framework scanned: NSA
┌────────────────┬────┐
│       Controls │ 19 │
│         Passed │ 7  │
│         Failed │ 12 │
│ Action Required │ 0  │
└────────────────┴────┘

Failed resources by severity:
┌──────────┬────┐
│ Critical │ 0  │
│     High │ 46 │
│   Medium │ 72 │
│      Low │ 17 │
└──────────┴────┘
```

Figure 16: Kubescape scan against code repo using NSA & MITRE Controls

As seen in Figure 16, the MITRE & NSA-specific scans show additional control and resource failures compared to the control.

The Kubescape VSCode plugin also revealed interesting findings. The plugin runs a scan whenever a file is saved in VSCode. It flagged 8 of the 10 vulnerability injects

when saving in VSCode. Surprisingly, the file scans did not flag two of the injects, K07 and K08. If only the VSCode plugin is used, two of these vulnerabilities could be pushed to the repository.

K07:Missing_Network_Segmentation_Controls
K08:Secrets_Management_Failures

While the remaining files were flagged as expected, several instances did not trigger alerts for the intended OWASP Kubernetes security vulnerabilities. For example, K02:Supply_Chain_Vulnerability did not flag a vulnerable container image (supply chain vulnerability). Users would need to use the **kubescape image scan** command to scan container images via Kubescape. However, it did flag Kubescape control C-0078 (Images from allowed registry). The control is intended to ensure that all container images used are sourced from the authorized repositories defined in Kubescape. Many other injects were flagged similarly, with the VSCode warning related, but not an exact match to the intended OWASP top 10 vulnerability.

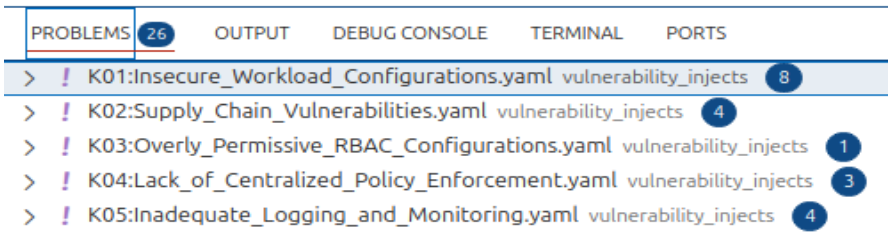



Figure 17: Example Kubescape VSCode plugin scan showing “Problems” after saving a file

Checkov scan against the repository

```
kubernetes scan results:

Passed checks: 566, Failed checks: 160, Skipped checks: 0
```

Figure 18: Checkov Kubernetes scan summary

The Checkov scan revealed additional failures. All vulnerability injects were identified except for one, which meant the tool flagged the YAML files. K07:Missing_Network_Segmentation_Controls has now bypassed both scans. As for the

rest of the flagged files, like the Kubescape scans, not all were identified for the exact OWASP Top 10 risk aimed to be replicated. For example, in one instance, the Checkov fail was generated for an unrelated reason. K03:Overly_Permissive_RBAC_Configurations.yaml failed only one Checkov test for using the default namespace. The failed check is unrelated to the overly permissive RBAC misconfiguration. Figure 19 shows the fail that alerted on K03.

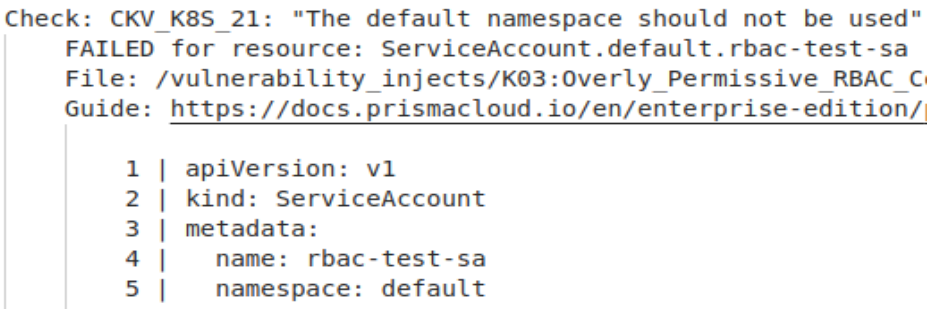



Figure 19: Checkov control failure revealing K03, not related to RBAC

The live cluster scans were identical to scenario one because the vulnerable injects were not applied to the cluster. This highlights that if only live cluster scans are used, all vulnerable code in the repository that is not applied to the cluster will go undetected.

Scenario two showed that repository scans are effective but imprecise. Kubescape and Checkov collectively identified most injected vulnerabilities, but often flagged the files for unrelated controls rather than our intended OWASP vulnerability. Using all tools for this testing flagged 9 out of 10 injects, but this scenario demonstrates that developer-side tooling alone should not be relied upon for complete protection.

## 4.3 Scenario 3 - Vulnerabilities injected into the repository and applied to the cluster

In scenario three, the vulnerability injections were located inside the repository and scans were conducted before deployment. Afterwards, all vulnerability injections were applied to the live Kubernetes cluster, and live cluster scans were performed.

As expected, the Kubescape and Checkov scans were identical to scenario 2 when scanning the repository and using the Kubescape VSCode extension to scan saved files.

When applying injections to the live cluster, all injections succeeded except one. K10:Outdated_and_Vulnerable_Kubernetes_Components.yaml could not be applied. Figure 20 shows the error message received when attempting to apply the inject.

```
(checkov-env) patrick@AWS:~/projects/SANS-White-Paper-EKS/vulnerability_injects
$ kubectl apply -f K10\:Outdated_and_Vulnerable_Kubernetes_Components.yaml
error: resource mapping not found for name: "k10-outdated" namespace: "" from "
K10:Outdated_and_Vulnerable_Kubernetes_Components.yaml": no matches for kind "D
eployment" in version "apps/v1beta1"
```

Figure 20: command revealing failure to apply K10

The API version apps/v1beta1 was removed in Kubernetes 1.16, so the outdated resource was used for the vulnerability. As a result, the cluster could not create the resource. Kubernetes (Version 1.34) prevented that injection from being applied.

The Kubescape security scan of the live cluster revealed many more warnings compared to Scenarios 1 and 2. Figure 21 shows the Kubescape scan results for the live cluster. Figures 22 and 23 show example results of drilling down into Kubescape control failures that identified the specific resource flagged.

Access control

| Control name | Resources | View details |
|---|---|---|
| Administrative Roles | 1 | $ kubescape scan control C-0035 -v |
| List Kubernetes secrets | 3 | $ kubescape scan control C-0015 -v |
| Minimize access to create pods | 1 | $ kubescape scan control C-0188 -v |
| Minimize wildcard use in Roles and ClusterRoles | 1 | $ kubescape scan control C-0187 -v |
| Portforwarding privileges | 1 | $ kubescape scan control C-0063 -v |
| Prevent containers from allowing command execution | 1 | $ kubescape scan control C-0002 -v |
| Roles with delete capabilities | 2 | $ kubescape scan control C-0007 -v |
| Validate admission controller (mutating) | 2 | $ kubescape scan control C-0039 -v |
| Validate admission controller (validating) | 1 | $ kubescape scan control C-0036 -v |

Secrets

| Control name | Resources | View details |
|---|---|---|
| Applications credentials in configuration files | 2 | $ kubescape scan control C-0012 -v |

Network

| Control name | Resources | View details |
|---|---|---|
| Missing network policy | 14 | $ kubescape scan control C-0260 -v |

Workload

| Control name | Resources | View details |
|---|---|---|
| Host PID/IPC privileges | 3 | $ kubescape scan control C-0038 -v |
| HostNetwork access | 1 | $ kubescape scan control C-0041 -v |
| HostPath mount | 3 | $ kubescape scan control C-0048 -v |
| Non-root containers | 8 | $ kubescape scan control C-0013 -v |
| Privileged container | 2 | $ kubescape scan control C-0057 -v |

Figure 21: Kubescape scan summary against live cluster

```
Kind: ServiceAccount
Name: rbac-test-sa
Namespace: default

Controls: 1 (Failed: 1, action required: 0)

┌──────────────────────────────────────────────────
│ Resources
├──────────────────────────────────────────────────
│ Severity              : Medium
│ Control Name          : Administrative Roles
```

Figure 22: Example drill-down: Kubescape alerting on K03 created service account for control C-0035, Administrative Roles

```
Kind: ConfigMap
Name: insecure-config
Namespace: default

Controls: 1 (Failed: 1, action required: 0)

┌──────────────────────────────────────────────────────────────────────────────
│ Resources
├──────────────────────────────────────────────────────────────────────────────
│ Severity             : High
│ Control Name         : Applications credentials in configuration files
```

Figure 23: Example drill-down: Kubescape alerting on K08, a configuration map with credentials

After reviewing the verbose scans of all flagged controls, the findings showed that all but one injection was flagged, K07 Missing Network Segmentation Controls. Notably, Kubescape scans performed against the live cluster more accurately identified the intended OWASP Kubernetes risks, whereas repository-based scans produced broader, less precise findings. For example, the scan of K03:Overly Permissive RBAC configurations identified the service account failing control C-0035 for Administrative Roles, and K08: Secrets Management Failure failed control C-0012 for application credentials in configuration files. Relying solely on repository scans, K03 was flagged in Checkov for not having the default namespace, and K08 was not flagged in the Kubescape VSCode scan for credentials.

Static repository scans detect structural misconfigurations, while live cluster scans detect actual security risks arising from Kubernetes object relationships and resolved configurations. Kubescape flagged injects in live mode because it evaluated effective RBAC, secrets usage, and workload behavior, whereas repository scans could only evaluate incomplete YAML without cluster context.

Another interesting finding came from K02:Supply_Chain_vulnerability. The Kubescape scan of the live cluster flagged this file for failing unrelated controls, such as Non-root containers (C-0013) and missing network policy (C-0260). To identify the supply chain vulnerability, we would need to use a different Kubescape scan designed for

images **kubescape scan image <image>**. Scanning the image with Kubescape's image-scanning command revealed the major security vulnerabilities in this image. The scan revealed 96 critical vulnerabilities in the container image as summarized below.

Image: vulhub/pgadmin:6.16

* 96 Critical
* 1098 High
* 2294 Medium
* 1754 Other

Using Kubescape to scan the specific file for the OWASP K07 injection YAML revealed no issues. The best explanation is that the configuration is not misconfigured or an outright error. It's just an extremely permissive security posture and is usually not considered best practice. Figure 24 shows the summarized results from Kubescape scanning the individual resource. Figure 25 reveals the YAML text of the resource, that creates a network policy with allow all permissions.

**kubescape scan K07:Missing_Network_Segmentation_Controls.yaml**

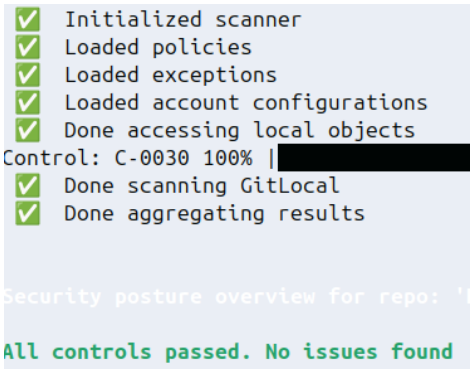



Figure 24:Kubescape scan results for the single file K07:Missing_Network_Segmentation_Controls.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: k07-no-network-controls
---
# Empty network policy = allows all
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all
  namespace: k07-no-network-controls
spec:
  podSelector: {}   # matches all pods
  ingress:
    - {}            # allows any ingress
  egress:
    - {}            # allows any egress
```

Figure 25:K07:Missing_Network_Segmentation_Controls.yaml

The MITRE and NSA compliance scores shown below showed a small decrease from Scenario 1 (MITRE: 75.71%, NSA: 71.42%).

- MITRE: 74.47%
- NSA: 64.69%

Kube-bench results are identical to those in Scenario 1, even with vulnerabilities injected into the cluster. This is because kube-bench does not scan workloads. It evaluates the Kubernetes control plane and node-level security settings against the CIS benchmarks (Aqua Security, n.d.).

Kube-Bench did not show any changes after the OWASP vulnerabilities were injected. This behavior is due to the use of Amazon EKS. In EKS, the control plane is fully managed by AWS and not accessible to the user, so Kube-Bench only performs Kubelet-level node checks. Because the injected vulnerabilities were workload-level issues (RBAC, insecure pods, secrets, network policies, supply-chain risks), Kube-Bench appropriately did not detect them. Kube-Bench checked 13 CIS controls in the worker

node security configuration section (12 passed and 1 failed), and the remaining WARNs indicate that the test requires further attention or manual investigation of the recommendations (Aqua Security, n.d.). This demonstrates the limitations of Kube-Bench in managed Kubernetes environments, such as EKS, for workload scanning. Amazon recommends using the tool (AWS, n.d.) for the automated scans it can perform. Figure 26 shows the summary of the scan that is identical to Scenario 1 and Figure 27 shows some of the different controls that Kube-Bench scanned against.

```
== Summary total ==
12 checks PASS
1 checks FAIL
33 checks WARN
0 checks INFO
```

Figure 26: Kube-Bench summary results

```
[INFO] 3 Worker Node Security Configuration
[INFO] 3.1 Worker Node Configuration Files
[PASS] 3.1.1 Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Automated)
[PASS] 3.1.2 Ensure that the kubelet kubeconfig file ownership is set to root:root (Automated)
[PASS] 3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)
[PASS] 3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)
[INFO] 3.2 Kubelet
[PASS] 3.2.1 Ensure that the Anonymous Auth is Not Enabled (Automated)
[PASS] 3.2.2 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)
[PASS] 3.2.3 Ensure that a Client CA File is Configured (Automated)
[PASS] 3.2.4 Ensure that the --read-only-port is disabled (Automated)
[PASS] 3.2.5 Ensure that the --streaming-connection-idle-timeout argument is not set to 0 (Automated)
[PASS] 3.2.6 Ensure that the --make-iptables-util-chains argument is set to true (Automated)
[FAIL] 3.2.7 Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture (Automated)
[PASS] 3.2.8 Ensure that the --rotate-certificates argument is not present or is set to true (Automated)
[PASS] 3.2.9 Ensure that the RotateKubeletServerCertificate argument is set to true (Automated)
[INFO] 4 Policies
[INFO] 4.1 RBAC and Service Accounts
[WARN] 4.1.1 Ensure that the cluster-admin role is only used where required (Automated)
[WARN] 4.1.2 Minimize access to secrets (Automated)
[WARN] 4.1.3 Minimize wildcard use in Roles and ClusterRoles (Automated)
[WARN] 4.1.4 Minimize access to create pods (Automated)
```

Figure 27: Example WARNs for best practices that would require manual intervention to check and implement, as opposed to the PASS/FAIL Kube-Bench results that were automated for worker node security configurations

Scenario three showed the highest detection fidelity once the Kubescape live cluster scans were incorporated. Runtime scanning enabled Kubescape to accurately identify workload behaviors, such as secret usage and RBAC permissions, that better align with our intended OWASP risks. The Kube-Bench results remained unchanged, highlighting

the limitations of the basic Kube-Bench job with workload-level vulnerabilities in an EKS environment.

## 4.4 Scenario 4 - Vulnerability injects not in the repository and applying security injects as “Configuration drift”

In scenario four, the study scanned a branch of the repository that did not contain the vulnerability injections prior to deployment. Later, all ten injects were applied to the live cluster, and then the live cluster scans were performed.

As expected, scanning the repository yielded the same results as in Scenario one, and scanning the live cluster yielded the same results as in Scenario three. This scenario highlights the risks of scanning only repository code and not the live cluster. All security vulnerabilities will go undetected if only an infrastructure-as-code scanning method is used.

Scenario four exposed a blind spot in limiting scans to IaC environments only. A repository scan would reveal none of the vulnerability injects, whereas live cluster scans uncovered 9 of the 10 injects. The scenario highlights the impact of configuration drift and the need to continuously validate the running cluster.

## 4.5 Scenario 5 – Vulnerability injects not in the repository with a GitHub workflow scanning commits

In the final scenario, the study created a GitHub workflow YAML file that contains multiple jobs. The jobs include a Checkov scan and a Kubescape scan, which will scan any file committed to the “scan” or “main” branch in the repository.

This scenario walks a fine line between security and production. These security tools seem to trigger alerts for numerous controls beyond our OWASP Top 10 targets. With that in mind, developers may push back against these potential production bottlenecks.

For testing purposes, the workflow silently alerts on flagged commits instead of blocking the commit. Figure 27 shows the initial code for the jobs that run the Checkov and Kubescape scans, and Figure 28 shows an example failure when pushing the K02 YAML file.

```
#
# 4. Run Checkov Scan
# ----------------------------------------------
- name: Run Checkov Scan
  if: ${{ steps.changes.outputs.iac_csv != '' }}
  id: checkov
  uses: bridgecrewio/checkov-action@v12
  with:
    file: ${{ steps.changes.outputs.iac_csv }}
    quiet: true
    soft_fail: true

6. Run Kubescape — Only Scan Real K8s Manifests
----------------------------------------------
name: Run Kubescape Scan
if: ${{ steps.changes.outputs.iac_csv != '' }}
run: |
  echo "Running Kubescape scans..."
  export PATH="${HOME}/.kubescape/bin:$PATH"
```

Figure 27: Jobs 4 and 6 in the GitHub workflow that start Checkov and Kubescape scans

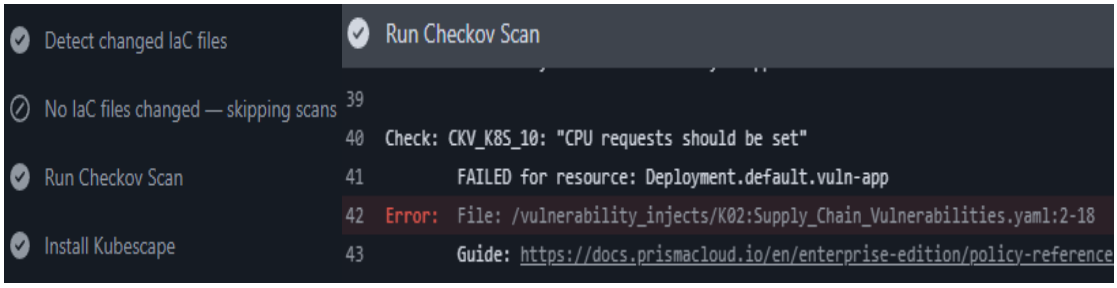



Figure 28: Some Git workflow jobs and an example flag from the K02 workflow

After pushing the files one at a time, all were flagged by either Checkov or Kubescape, except K07:Missing_Network_Segmentation_Controls.yaml.

```
Security posture overview for repo: 'vulnerability_injects/K07:Missing_Network_Segmentation_Controls.yaml'

All controls passed. No issues found
```

Figure 29: K07 allowed to be pushed through the workflow with no control failures

Two more anomalies were noted. When the job ran against K10:Outdated_and_Vulnerable_Kubernetes_Components.yaml, Checkov was able to run the scan and flag the file, but Kubescape failed to complete the scan. Additionally, the Kubescape job flagged K08:Secrets_Management_Failures.yaml, while the Kubescape VScode plugin did not.

The Kubescape live cluster scan revealed no vulnerability injects, as 9 of the 10 were assumed not to have been merged into the repository.

Scenario 5 demonstrated that incorporating Kubescape and Checkov scans into an automated workflow greatly improves early detection. Nine out of ten vulnerability injections were flagged and assumed to be prevented from merging into the repository.

# 5. Recommendations and Implications

This section translates the findings from the five experiment scenarios into practical security recommendations for securing Kubernetes environments. The findings revealed differences in detection accuracy and coverage across scanning approaches. The recommendations are derived from observed strengths and limitations of repository-based scans, live cluster runtime scans, and CI/CD scanning jobs in detecting Kubernetes security misconfigurations. To support these recommendations, Figure 30 summarizes detection outcomes across all five scenarios, displaying how each scanning approach performed as vulnerability injects were introduced.

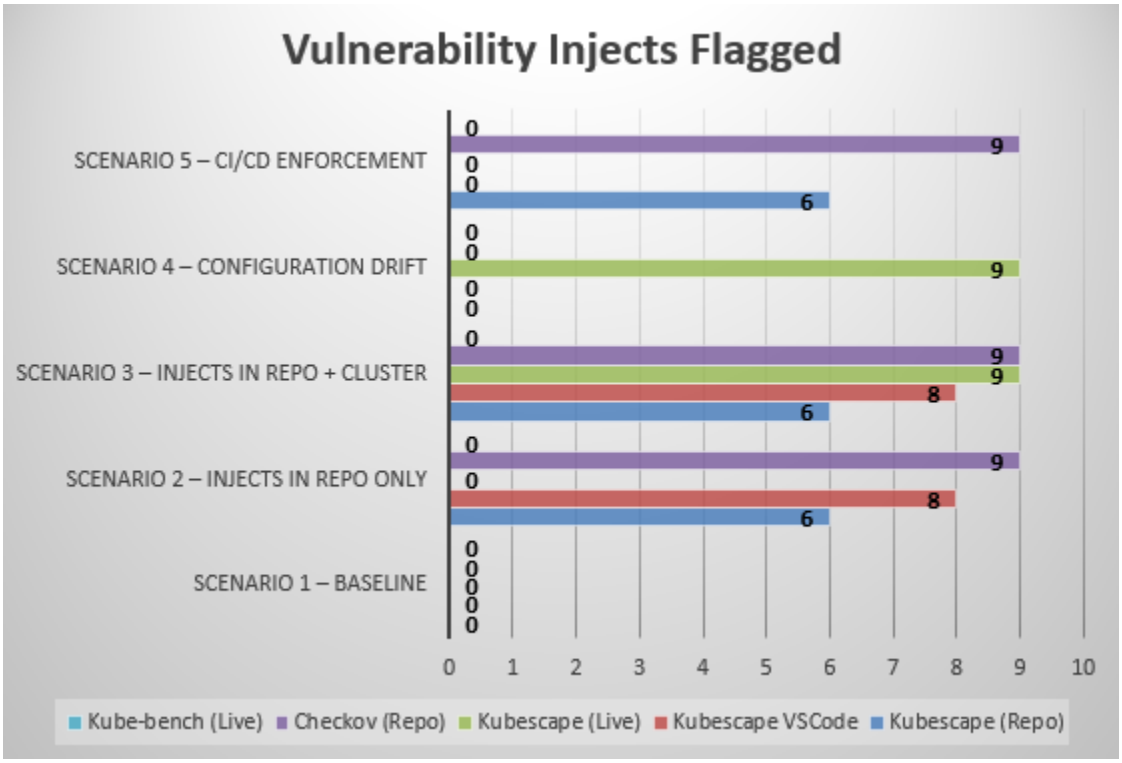



Figure 30: Summarized results from five test scenarios

## 5.1 Image Security Scanning

Starting from the source, the container images used in the cluster should be scanned for vulnerabilities. Kubescape provides this capability, but a specific Kubescape command for image scanning must be run. This check could be implemented before committing Kubernetes resources to the code repository. For each commit, set up a Git workflow to identify container images in a resource file and scan them with Kubescape. This method would have revealed the OWASP risk for the inject K02:Supply_Chain_Vulnerability.yaml.

## 5.2 CI/CD and developer feedback loop

Git workflows for the IaC repository can be set up by creating a YAML file with jobs in the .github/workflows directory. Scenario 5 demonstrated that integrating Kubescape and Checkov jobs improved early detection of insecure code. These jobs will

reduce the likelihood that insecure configurations will be merged into the repository. A caveat to this approach is a potential slowdown in production, as the tools generate a high volume of alerts from other control failures. A risk-based approach is recommended, where findings generate warnings rather than blocking commits, allowing teams to refine policies over time.

## 5.3 Repository (IaC) Scanning

Repository-based scans using Kubescape and Checkov successfully flagged most injected vulnerabilities before deployment. The tools frequently flagged files for unrelated controls that were not aligned with the intended OWASP risk, suggesting these tools can be imprecise at identifying risks related to Kubernetes workloads. Despite this limitation, IaC scanning remains a useful and critical preventive control, recommended before deploying the cluster.

## 5.4 Live Cluster Scanning and Runtime Validation

Regarding the correct identification of injected OWASP vulnerabilities, the live Kubescape scan was the most successful. The tool flagged 9 of 10 files and performed best at matching each injection to its corresponding OWASP vulnerability. Kubescape's runtime analysis evaluated permissions, resolved configurations, and actual resource relationships that static repository scans could not fully capture. Security best practices recommend scanning the Kubernetes environment for vulnerabilities at least daily to rapidly detect and resolve issues (Upwind.io, n.d.).

## 5.5 Understanding Tool Scope

Understanding a tool's scope is critical to correctly interpreting scan results. Without knowing a tool's purpose and intent, there can be a false sense of security. The study observed this phenomenon with the Kube-Bench tool on the live cluster and with

the IaC scanning tools, which had difficulty pinpointing the specific workload vulnerability the injects were attempting to recreate.

# 6. Conclusion

This research shows that Kubernetes security requires a multi-dimensional approach. IaC scanning alone cannot detect runtime risks, and live scanning without repository analysis leaves organizations blind to deployment intent. Configuration drift adds further complexity, enabling vulnerabilities to emerge even when code repositories appear clean.

Ultimately, securing Kubernetes environments requires:

- Image security scans
- Continuous scanning across development, deployment, and runtime
- Integration of tooling into CI/CD processes
- Regular compliance validation
- Awareness of tool limitations, especially in cloud-managed clusters

A layered, defense-in-depth strategy is essential for addressing the OWASP Kubernetes Top 10 and for maintaining a resilient Kubernetes infrastructure.

# References

Amazon Web Services. (n.d.). *Amazon EKS best practices: Protecting the infrastructure*. Retrieved from https://docs.aws.amazon.com/eks/latest/best-practices/protecting-the-infrastructure.html

Aqua Security. (n.d.). *kube-bench flags and commands (v0.6.7)*. Retrieved from https://aquasecurity.github.io/kube-bench/v0.6.7/flags-and-commands

Basak, S. K., Neil, L., Reaves, B., & Williams, L. (2023). *SecretBench: A dataset of software secrets*. arXiv preprint arXiv:2303.06729.

Center for Internet Security. (n.d.). *CIS critical security controls*. https://www.cisecurity.org/controls

Cloud Native Computing Foundation. (2024). *Cloud Native 2024 annual survey* (Report). Linux Foundation Research.

DZone, Inc. (2020). *DZone Kubernetes bundle (Refcard)*. Retrieved from https://dzone.com/storage/attachments/14131598-dzone-kubernetesbundle.pdf

Eleweke, I. (2025, October 14). *Fortifying AI infrastructure: Securing code, configuration, and integrity in national systems*. *International Journal of Advance Research, Ideas and Innovations in Technology*, 11(5). https://www.ijariit.com/manuscript/fortifying-ai-infrastructure-securing-code-configuration-and-integrity-in-national-systems/

HashiCorp, Inc. (n.d.). *Terraform documentation*.

https://developer.hashicorp.com/terraform

Hohn, A. (2018). *Getting started with Kubernetes*. DZone.

https://dzone.com/storage/attachments/14131598-dzone-kubernetesbundle.pdf

Minna, F., Blaise, A., Tuma, K., & Massacci, F. (2025). *Automated analysis of security policy violations in Helm charts*. *IEEE Transactions on Dependable and Secure Computing*.

Mocevicius, R. (2020). *kubectl: Command-line Kubernetes in a nutshell: Deploy, manage, and debug container workloads using the Kubernetes CLI*. Packt Publishing Ltd.

OWASP Foundation. (n.d.). *OWASP Kubernetes Top 10*. https://owasp.org/www-project-kubernetes-top-ten/

OWASP Foundation. (2025). *OWASP Kubernetes Top Ten*. https://owasp.org/www-project-kubernetes-top-ten/

Pillai, T. N. K. (2024, October 7). *Kube-bench: A comprehensive guide*. Medium. https://medium.com/@thamunkpillai/kube-bench-a-comprehensive-guide-14cc022a3e8c

Red Hat. (2023). *Kubernetes adoption, security, and market trends: An overview of the state of Kubernetes security report*. Red Hat. https://www.redhat.com/en/resources/kubernetes-adoption-security-market-trends-overview

Shwartz, I. (2025, March 26). *Drift away: The hidden risk of large-scale Kubernetes*

*environments*. Komodor. https://komodor.com/blog/drift-away-the-hidden-risk-of-large-scale-kubernetes-environments/

Upwind.io. (n.d.). *What is Kubernetes vulnerability scanning?* Retrieved from https://www.upwind.io/glossary/what-is-kubernetes-vulnerability-scanning (security best practices recommend scanning at least daily)

Wilson, J., Diachkova, O., & Floyd, R. (2025, July 10). *The 2025 Docker state of application development report*. Docker. https://www.docker.com/blog/2025-docker-state-of-app-dev/

Wiz, Inc. (n.d.). *Top Kubernetes security tools*. https://www.wiz.io/academy/container-security/top-kubernetes-security-tools